Distributed Streaming Collaboration

Network Application Development - Technical Explanation Report

Implementation: Relay / network_app. Python 3.11+ raw sockets; verified locally on Python 3.12. The official ten-milestone rubric guides this educational trusted-LAN implementation.

1. System architecture

```
Browser A -- HTTP loopback --> Client A
                                    | TCP admission
                                    | encrypted UDP records
                               Central server
                                    | encrypted UDP records
                                    | TCP admission
Browser B -- HTTP loopback --> Client B
```

The central server owns nickname/session state, source IP and active TCP/UDP ports, and routes global/private records. Every client owns its own transport, local file streams and bounded event history. The HTTP bridge is a separate loopback-only process with a random capability token. Static HTML/CSS and vanilla JavaScript poll that bridge; distributed application traffic never uses HTTP.

The assignment contains a TCP-transfer example but explicitly grades UDP file transfer. This implementation uses TCP for HELLO/AUTH/AUTH_OK, session lifetime and logout; UDP handles messaging, presence, heartbeats and all file bytes. No high-level networking framework or WebSocket is used.

Concurrency uses a thread per TCP session and a UDP receive/retransmission reactor plus application dispatch worker. Twenty simultaneous clients and simultaneous broadcasts are covered by real-socket tests. Server state is protected with a reentrant lock.

Lifecycle: CONNECTED -> HELLO -> AUTH -> ACTIVE -> LOGOUT/EOF/INACTIVITY -> CLOSED. Duplicate active nicknames are rejected. Nickname admission is not a persistent password account system.

2. Custom protocol specification

TCP framing: unsigned 32-bit big-endian length followed by UTF-8 JSON, maximum 16,384 bytes. HELLO declares version 1. AUTH carries username, UDP port and X25519 public key. AUTH_OK returns session UUID, ephemeral server key and UDP port; AUTH_ERROR reports rejection.

```
UDP fixed header: !2sB16sQ (27 bytes)
magic NC | kind 0=DATA,1=ACK | session UUID | counter
body: AES-GCM ciphertext + 16-byte tag
maximum complete datagram: 1200 bytes
```

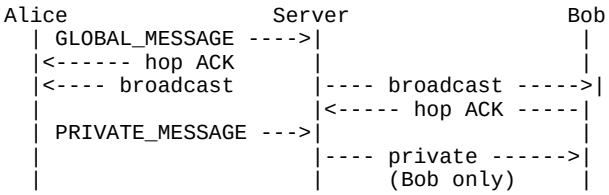
DATA plaintext is compact JSON. ACK plaintext is an eight-byte packet counter. Packet length follows the datagram boundary; NC identifies version 1. Header bytes are authenticated additional data. Counters form 12-byte nonces. Directional AES-GCM keys prevent counter collisions across directions. Retransmission resends the identical packet. A bounded 8192-counter replay window suppresses duplicates.

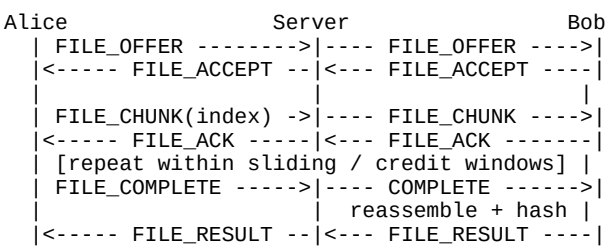
Application types: READY, USER_LIST, HEARTBEAT, GLOBAL_MESSAGE, PRIVATE_MESSAGE, ERROR, FILE_OFFER, FILE_ACCEPT, FILE_ACK, FILE_CHUNK, FILE_COMPLETE and FILE_RESULT. Messages have id/text/recipient with server-stamped sender/time. Offers have id/name/size/SHA-256; chunks add index and base64 payload; results add success and computed hash. Presence snapshots carry a monotonic revision.

A file UUID and sender identify its logical stream. Chunks carry 512 binary bytes before base64 encoding. Files are capped at 1 GiB (32 MiB browser upload). Temporary files receive indexed writes; unique byte counts and SHA-256 gate atomic finalization. Duplicate chunks do not inflate progress. An early completion waits for missing chunks. Stale partial transfers expire and are deleted.

Selective-repeat windows begin at four packets and cap at 32. Missing ACKs trigger 150 ms initial retransmission timeout with exponential backoff capped at two seconds; twelve retries signal failure. A separate sixteen-chunk receiver-credit limit bounds per-file relay pressure. Chat records do not wait for file sequence gaps; chat itself is arrival ordered.

3. Message and file sequences





Packet ACKs operate independently on each encrypted hop. FILE_ACK credits reflect receipt at the destination. FILE_RESULT is the end-to-end integrity outcome. Local private message echo is not a read receipt. Offline or congested recipients produce errors.

Congestion handling is educational CUBIC-inspired window growth: target = 0.4*(elapsed-K)^3 + Wmax, K = (0.3*Wmax/0.4)^(1/3), combined with additive ACK growth. Timeout reduces the window by 0.7 and restarts the epoch. It is not full CUBIC or BBR: pacing, adaptive RTT estimates, standards-level fairness and fast recovery are absent.

X25519 and HKDF establish session secrets; AES-GCM protects UDP content and headers. The server decrypts routed data. No pinned certificate or identity authenticates the initial key exchange, so active man-in-the-middle attacks remain possible. Use a trusted LAN/VPN. TCP metadata and packet sizes remain visible.

4. Challenges and solutions

The assignment combines a UDP streaming objective with a TCP file-transfer example. Following the explicit UDP file-transfer milestone resolves that conflict: TCP provides admission and session lifecycle, while UDP carries every distributed message and file record. The browser cannot use raw UDP, so a loopback HTTP bridge owns a real socket client. This preserves the network learning objectives while satisfying the HTTP/JavaScript UI milestone.

UDP reliability requires more than resending entire files. Each encrypted datagram has a unique counter, selective ACK, bounded in-flight window and retry deadline. A delayed or lost chunk therefore causes retransmission of that packet only. Out-of-order chunks write directly to their indexed disk positions. Completion may arrive early, so the receiver remembers completion until its unique-byte count reaches the advertised size. SHA-256 then decides whether the temporary file may be finalized. A test sends intentionally incorrect metadata hashes and verifies that no successful file survives.

Reliable hop delivery does not itself protect relay buffers from a fast sender and slow receiver. An additional receiver-credit window restricts each file to sixteen unacknowledged chunks. This limits queued relay work without loading files into memory. It also leaves chat records free to proceed without waiting for a missing file chunk. Shared congestion windows and dispatch workers still introduce scheduling contention; the implementation makes no claim of unlimited stream isolation or guaranteed chat order.

Encryption adds a subtle nonce invariant. Both directions could otherwise generate the same counter under the same key. Distinct client-to-server and server-to-client keys solve that issue, while retransmission reuses the original complete ciphertext rather than encrypting different data under the same nonce. Header authentication binds ciphertext to the session, packet kind and counter. An 8192-counter deduplication window prevents packet replay from creating duplicate chat/file events. The unauthenticated X25519 exchange protects against passive sniffing but not active impersonation, a deliberate trusted-LAN limitation.

Threaded cleanup was an actual integration finding: a transport could enter its send tick after another thread closed the socket. Shutdown now changes state under the transport lock and the reactor checks it before sending again. Warnings-as-errors also identified unclosed read handles in tests, which were replaced by managed path reads. These checks matter because a superficially successful file transfer can conceal background-thread failures and resource leaks.

The test impairment adapter lives only under tests. Deterministic loss, duplication, reordering and latency exercise real UDP sockets rather than fake application callbacks. The large-file test runs concurrently with chat, and the concurrency test establishes twenty sessions before simultaneous broadcasts. These demonstrate the intended educational workload; they do not establish production Internet scalability, fairness, hostile-client resistance, or automatic reconnection.

5. Validation and practical limits

Automated tests exercise real localhost TCP/UDP sockets, not only mocks. Coverage includes Unicode and binary framing, invalid lengths, nickname admission, duplicate login, presence, global/private isolation, twenty concurrent sessions, file integrity and concurrent chat, replay and tampering, abrupt disconnect, inactivity, retry exhaustion and HTTP bridge authorization.

File sizes include zero bytes, 37 bytes, 65,539 bytes and 10,485,791 bytes. A separate 80,000-byte test uses deterministic outbound loss every eleventh packet, duplicates every fifth, holds every seventh for reordering, and adds one millisecond latency. Success requires a receiver hash match and observed retransmissions.

Actual final pytest results are recorded in docs/testing.md alongside the raw pytest output. The implementation was iteratively tested; an initial successful functional run exposed shutdown-thread warnings, and stricter warning checks drove resource cleanup fixes. No manual multi-machine or full browser interaction result is claimed.

Run: python -m pytest -q -W error. Start server: python -m network_app.server. Start three client bridges with python -m network_app.webui --port 8081, 8082 and 8083 in separate terminals. Open each full token URL and log in as Alice, Bob and Charlie. README supplies detailed installation, LAN firewall and demonstration steps.

Remaining limits: no durable identities, offline history, transfer resume, NAT traversal or hostile-client resource defense. Shared windows still permit scheduling contention. Browser uploads retain spool files for inspection. Presence supports the required twenty-client scale, not arbitrary large rosters. Congestion control and authentication remain educational simplifications; manual browser and cross-platform LAN validation should precede submission.